



Trust Skills, Not Portfolios.

Scora - Technical Research Notes

Architecture trade-offs, trust design, and secure execution constraints

Technical folder

Product: Scora Team: Code Luck

Phase: Product Validation & Strategy Date: 27 July 2026

Key technical findings

- The core application can begin as a modular monolith; untrusted code execution must be isolated as a separate security boundary from day one.
- Verification is not a weighted score alone. Mandatory ownership gates - explanation, hidden-bug diagnosis, and live modification - must pass before the track can become verified.
- Use two explicit assessment modes: an AI-restricted fundamentals checkpoint and an AI-allowed engineering task. Tool usage is declared and governed, not guessed from unreliable detection.
- Deterministic tests and static analysis measure the artifact; code-grounded interviews and live changes measure whether the person can own it; human review resolves ambiguity and appeals.
- A Trust Score needs evidence lineage, model/rubric versions, confidence, recency, gate outcomes, appeal history, and calibration data.
- Behavioral integrity signals trigger review rather than automatic guilt. Scora excludes unsupported skill claims from verified discovery without publicly accusing users of cheating.

Architecture decisions

Decision	MVP choice	Revisit when
Backend shape	Modular monolith + workers.	Teams or scaling needs create independent release boundaries.
Code isolation	Dedicated hardened workers with ephemeral containers.	Before public scale or support for higher-risk languages/packages.
AI provider	API abstraction with versioned prompts and evaluations.	Volume, cost, privacy, or quality justifies specialization.
Scoring	Rule-based weighted components + confidence metadata.	Calibration data supports statistical or learned models.
Verification gate	Required explanation, debugging, and live-change rubrics before scoring.	Calibration demonstrates a better gate or role-specific thresholds.
Human review	Sampled + triggered + appealed.	Measured error rate identifies a different optimal review policy.
Marketplace	Discovery and shortlist only.	Validated demand requires contracts, escrow, or payment operations.

Security acceptance checklist

- No privileged containers, Docker socket, host root mounts, or application secrets.
- Non-root execution with CPU, memory, process, log, file, and time limits.
- Network disabled by default; allowlist exceptions are challenge-versioned.
- Ephemeral workspace deleted after retention and audit requirements are met.
- Signed minimal images, fixed language versions, patch process, and dependency policy.
- Central audit events, execution trace IDs, incident runbook, and regular sandbox testing.

References

We selected sources for their direct relevance and authority. Company pages describe current product behavior; surveys and research papers support market and risk claims. Assumptions that still require interviews or field testing are identified in the validation sections.

1. Stack Overflow 2025 Developer Survey - AI. Among respondents answering the accuracy question, 46% distrust AI-tool accuracy versus 33% who trust it. The most reported frustration is output that is almost right, while 45% cite extra time debugging AI-generated code.
<https://survey.stackoverflow.co/2025/ai>
2. GitHub randomized controlled study on Copilot and code quality. A study with 202 developers who had at least five years of experience found improved functionality and modest quality gains with Copilot. AI use alone is therefore not evidence of low skill.
<https://github.blog/news-insights/research/does-github-copilot-improve-code-quality-heres-what-the-data-says/>
3. Do Users Write More Insecure Code with AI Assistants?. In a security-focused user study, participants with an AI assistant produced less secure code and were more likely to believe their code was secure; lower trust and more active engagement correlated with fewer vulnerabilities.
<https://arxiv.org/abs/2211.03622>
4. Lost at C - USENIX Security 2023. A different user study found only a small security impact in its specific C task. This mixed evidence shows that risk depends on the task, user, and verification process rather than AI use alone.
<https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval>
5. METR - Early-2025 AI and experienced developer productivity. A randomized study of experienced open-source developers working in familiar repositories found early-2025 AI tools slowed completion in that setting, despite participants expecting a speedup.
<https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>
6. Generative AI code tools and software-engineer hiring. A study of 32 industry professionals reports that GenAI complicates evaluation of candidates' true abilities, while many participants value candidates who can demonstrate effective and informed AI use.
<https://arxiv.org/abs/2409.00875>
7. OWASP Secure Coding with AI. OWASP recommends independent verification, full-diff review, adversarial testing, dependency checks, human accountability, and sandboxed execution for AI-assisted development.
https://cheatsheetseries.owasp.org/cheatsheets/Secure_Coding_with_AI_Cheat_Sheet.html
8. Upwork Job Success Score. Upwork's Job Success Score reflects client satisfaction, contract outcomes, and long-term client relationships. It is a reputation signal rather than a direct test of code ownership or current engineering understanding.
<https://support.upwork.com/hc/en-us/articles/38437362753171-What-is-a-Job-Success-Score>
9. Upwork - Build a complete profile. Upwork emphasizes work history, certifications, availability, achievements, keywords, and portfolios as profile trust inputs.
<https://support.upwork.com/hc/en-us/articles/34924882793107-Build-a-100-complete-profile>
10. HackerRank Screen - AI-era skills and integrity. HackerRank separates fundamentals without AI, ability to work with AI, and AI concepts, while adding identity and assessment-integrity controls. This validates multi-mode assessment in the AI era.
<https://www.hackerrank.com/products/screen>
11. Codility Task Library. Codility combines real-life skills, problem solving, and technical knowledge tasks, supporting a rounded assessment rather than a single puzzle score.
<https://support.codility.com/hc/en-us/articles/360043827713-The-Codility-Task-Library>
12. Docker Engine security. Docker documents namespaces, cgroups, capabilities, daemon attack surface, and hardening controls relevant to isolated code execution.
<https://docs.docker.com/engine/security/>